DAG-EVM and Conflict-Graph Consensus for the Lux Multi-Chain

Zach Kelling

Lux Industries

March 2026

Abstract

We present a unified DAG consensus model for the Lux multi-chain that lifts Block-STM-style optimistic concurrency from a per-block scheduler into a first-class finality primitive. Each block-producing chain (C-Chain EVM, D-Chain DEX, Z-Chain ZK-UTXO, A-Chain AI, O-Chain Oracle, R-Chain Relay, S-Chain ServiceNode) emits transaction batches as vertices in a native directed acyclic graph, with conflict sets derived from the chain's natural state model: EVM read/write sets, UTXO nullifiers, DEX order tuples, job identifiers, feed rounds, destination-chain nonces, or service epochs. Antichains are finalised in parallel under the Quasar post-quantum certificate scheme; sequential worldview is preserved by a serialisability theorem. We formalise the safety and no-double-spend properties in Lean 4 and TLA$^+$, prove commutativity of non-conflicting vertices, and show that GPU-native conflict-graph reduction plus GPU ecrecover/keccak yields a 32× practical speedup on the dominant critical-path operations. The result is a DAG-EVM that preserves bytecode-level compatibility with Ethereum, a DAG-Z-Chain that preserves ZK-UTXO double-spend security, and a framework that cleanly contrasts Lux against contemporary parallel and DAG blockchains (Aptos, Sui, Solana, Monad, Fuel).

Contents

1 Introduction

Parallel blockchain execution splits into two orthogonal problem families: *intra-block* parallelism (execute the transactions of a linear block in parallel) and *inter-block* parallelism (finalise non-conflicting blocks in parallel, a partial-order consensus). The industry has focused almost exclusively on the first: Aptos' Block-STM [1], Monad's pipelined execution [3], Sei's parallel EVM, and Polygon Miden's optimistic execution all run in linear-chain consensus. DAG consensus, when present (Sui's Mysticeti [2], Aleph Zero's AlephBFT, Avalanche's Snowman^{++}), typically does not thread the DAG into the state-machine layer: blocks are ordered by the DAG and then executed sequentially, so the DAG exists above the execution engine rather than inside it.

Lux takes the third path: *the execution engine's conflict graph **is** the consensus DAG.* Each VM emits vertices whose parents are determined by the state-level dependency relation of that VM, and the DAG engine (nebula) finalises antichains that by construction contain no mutually-conflicting vertices. The downstream property is **serialisability by construction**: any topological order of the accepted DAG yields identical state, and no two finalised siblings can double-spend, re-write the same EVM slot, or re-submit the same nullifier.

This paper presents:

- A uniform definition of the per-chain conflict rule (§2);
- The DAG-EVM construction and its Block-STM lineage (§3);
- The DAG-Z-Chain nullifier-conflict construction (§4);
- Lean 4 formal proofs of safety, commutativity, and serialisability (§5);
- TLA$^+$ specifications and model-checked invariants (§6);
- A principled comparison with contemporary parallel and DAG blockchains (§7);
- A discussion of GPU-native EVM execution covering the v0.20.0 $\rightarrow$ v0.23.0 release cycle, including spec-complete Metal and CUDA opcode kernels, the EVMC host bridge for `CALL`/`CREATE` pre-resolution, and a unified pipeline that runs EVM execution, consensus state hashing, and post-quantum signature verification on a single Metal device (§8);
- A four-way parity test that runs the same 133-vector bytecode corpus through CPU_Sequential, CPU_Parallel (Block-STM), GPU_Metal and GPU_CUDA, asserting bit-exact agreement on `(gas_used, status, output)` (§10);
- The CUDA port methodology, three consensus-relevant bugs surfaced by the differential-oracle property of cross-implementation parity, and what 90.87% line / 100% function LLVM source-based coverage on the spec witness contributes to the production-readiness argument (§11);
- Empirical results across all four backends on the workloads above (§12).

2 Chain-Level Conflict Rules

Every VM in the Lux fleet exposes a single predicate, $\mathsf{conflicts}(v_1, v_2) \in \{\mathsf{true}, \mathsf{false}\}$, over pairs of vertices. The predicate must be *symmetric* and *computable from the vertex batch alone.* Table 1 summarises the per-chain rule.

We say a pair of vertices *commutes* iff it does not conflict.

Chain	Symbol	Engine	Conflict key set	Source
C-Chain (EVM)	C	DAG	$R(v) \cup W(v) \subseteq \mathsf{StorageKey}$	Block-STM
D-Chain (DEX)	D	DAG	$\{(base, quote, side, id)\} \subseteq \mathsf{OrderKey}$	orderbook
X-Chain (UTXO)	X	DAG	$\{\mathsf{txo}_i\} \subseteq \mathsf{UTXO}$	UTXO inputs
Z-Chain (ZK-UTXO)	Z	DAG	$\{\mathsf{null}_i\} \subseteq \mathsf{Nullifier}$	nullifier reveal
Q-Chain (PQ)	Q	DAG+Quasar	$\{\mathsf{stamp}_i\}$	PQ attestation
A-Chain (AI)	A	DAG	$\{\mathsf{jobID}_i\}$	job graph
O-Chain (Oracle)	O	DAG	$\{(\mathsf{feed}, \mathsf{round})\}$	feed rounds
R-Chain (Relay)	R	DAG	$\{(\mathsf{dstChain}, \mathsf{nonce})\}$	relay nonce
S-Chain (ServiceNode)	S	DAG	$\{(\mathsf{node}, \mathsf{epoch})\}$	service epoch
P-Chain (Platform)	P	Linear	ordered	validator set
B-Chain (Bridge)	B	Linear	ordered	ceremony steps
T-Chain (Threshold)	T	Linear	ordered	FROST rounds
M-Chain (MPC)	M	Sharded-linear	per-wallet linear	CGGMP21 rounds
K-Chain (Keys)	K	Linear	ordered	key rotation
I-Chain (Identity)	I	Linear	ordered	identity updates

Table 1: Per-chain conflict rule. The first block operates under the DAG engine with native parallel finality. The second block retains linear ordering where the state machine requires total order (validator registry, cryptographic ceremonies, identity updates).

Definition 1 (Conflict DAG)**.** A *conflict DAG* $D = (V, E, \mathsf{conflicts})$ is a directed acyclic graph whose vertices are VM-specific transaction batches, whose edges are causal parent relationships, and whose conflict predicate satisfies the *Siblings-Commute invariant*: for every pair of vertices v_1, v_2 with $v_1 \not\preceq v_2$ and $v_2 \not\preceq v_1$ in the DAG partial order, $\mathsf{conflicts}(v_1, v_2)$ is false.

The Siblings-Commute invariant is enforced constructively by the VM's BuildVertex: when forming a new vertex the builder scans the current *frontier* (set of rank-maximal accepted vertices) and refuses any batch whose conflict key set overlaps a frontier sibling. This is an $O(|\text{frontier}| \cdot |\text{keys}|)$ bitmap intersection; on GPU it is a single threadgroup-reduce kernel.

3 DAG-EVM

3.1 Lineage: Block-STM Lifted

Block-STM [1] (Aptos Labs, 2022) is an optimistic concurrency control (OCC) algorithm that speculatively executes a linear block's transactions in parallel on separate copies of state, tracks per-transaction read and write sets, and re-executes any transaction whose reads were invalidated by an earlier-committed transaction's writes.

Block-STM's correctness reduces to two properties:

1. Every execution is *conflict-equivalent* to the sequential execution of the same block in the block's canonical order.

2. Every committed write is visible to every strictly-later transaction that reads the same storage location.

These properties are purely local to the block's set of transactions; they make no reference to the fact that blocks are arranged in a linear chain. This is the opening we exploit.

3.2 Construction

Let T be a sequence of pending EVM transactions. The DAG-EVM builder performs:

Algorithm 1 BuildVertex (DAG-EVM)

1: Drain up to N txs from the mempool into T
2: Run Block-STM speculative exec on T over the current state snapshot
3: Emit $(R, W) \leftarrow$ union of per-tx read/write sets
4: $P \leftarrow$ minimal antichain of frontier vertices whose W covers R
5: Let $v.\mathsf{parents} \leftarrow P$
6: $v.\mathsf{id} \leftarrow \mathsf{keccak256}(\mathsf{txHashes} \parallel R \parallel W)$
7: Emit v to the DAG engine

The DAG engine (nebula) votes on acceptance. Once an antichain at rank r is fully populated with accepted vertices, Quasar stamps it with a post-quantum triple-signature certificate and r becomes final. The EVM state is updated by applying the vertices of each finalised antichain in any valid topological order.

3.3 Correctness Sketch

Theorem 2 (Non-Conflict Commutativity, Lean: `nonConflict_commute`)**.** *For any two EVM vertices v_1, v_2 with $\mathsf{conflicts}(v_1, v_2) = \mathsf{false}$ and any state σ:*

$$v_2.\mathsf{apply}(v_1.\mathsf{apply}(\sigma)) = v_1.\mathsf{apply}(v_2.\mathsf{apply}(\sigma)).$$

Proof sketch. By case analysis on membership of each storage key k in the write sets of v_1 and v_2. The non-conflict hypothesis excludes write-write overlap; the footprint lemmas (`writesOnly`, `readsOnly`) in the Lean development handle the remaining four cases. Full proof in `Consensus/DAGEVM.lean:60--1`
□

Theorem 3 (Topological Order Invariance, Lean: `topo_equivalence`)**.** *Given the Siblings-Commute invariant, any two topological orderings L_1, L_2 of the same accepted DAG D satisfy $\mathsf{applyList}(L_1, \sigma) = \mathsf{applyList}(L_2, \sigma)$ for every initial state σ.*

Theorem 4 (No Double Spend, Lean: `no_double_spend`)**.** *If v, w are both accepted, $v \neq w$, and both write the same storage key k, then v and w must be in distinct ranks (one is an ancestor of the other in the DAG).*

Corollary 5 (Serialisability)**.** *Every DAG-EVM execution is conflict-equivalent to a sequential EVM execution over the same set of transactions. An external observer sees a linear history of state transitions; the underlying parallel vertex topology is invisible at the state projection.*

4 DAG-Z-Chain

The Z-Chain operates the classical ZK-UTXO model: each transaction reveals a set of nullifiers (the anonymous counterparts of spent UTXOs), creates new commitments, and carries a ZK proof (Groth16 or Plonk) that the nullifier/commitment relationship is valid under a known verification key. The Z-state is the pair $(\mathsf{spent}, \mathsf{committed})$ with both components monotone (never shrink).

The conflict rule is dramatically simpler than in the EVM:

$$\mathsf{conflicts}(v_1, v_2) \iff N(v_1) \cap N(v_2) \neq \emptyset$$

where $N(v)$ is the union of nullifier sets across the transactions of v.

Theorem 6 (Z-Chain No Double Spend, Lean: `no_double_spend`)**.** *No nullifier appears in two distinct accepted vertices of the Z-Chain DAG.*

Theorem 7 (Batch Verification Soundness, axiom: `batch_sound`)**.** *Groth16 (and Plonk) batch verification is sound iff each constituent proof is individually sound. GPU-accelerated batch verify preserves this property under the pairing-based (resp. polynomial-commitment) assumption.*

Theorem 8 (FHE Commutativity, Lean: `fhe_linearity_preserved`)**.** *If two Z-Chain vertices each perform an additive FHE balance update on disjoint nullifier keys, the two updates commute.*

These together give DAG-Z-Chain the same parallelism profile as DAG-EVM while preserving all of the ZK-UTXO security invariants, *without* additional cryptographic assumptions beyond those already underlying the proof system.

5 Lean 4 Development

The full development is in `lux/formal/lean/Consensus/`:

- `DAGEVM.lean` — DAG-EVM safety, commutativity, topological order invariance, no-double-spend, serialisability.
- `DAGZChain.lean` — DAG-Z-Chain nullifier-disjointness, proofs-commute, parallel-verify soundness, FHE linearity.
- `PQZParallel.lean` — cross-chain non-interference: running C/X/Z/Q/D/A/O/R/S DAGs concurrently preserves per-chain safety; Quasar stamps are independently verifiable; validator signing across k chains scales linearly.
- `Safety.lean`, `Liveness.lean`, `BFT.lean` — Pre-existing per-chain consensus safety (not modified).
- `Quasar.lean` — PQ certificate unforgeability (not modified); supplies the finality assumption for DAG antichains.

Build: `cd lux/formal/lean && lake build`.

6 TLA$^+$ Specifications

The TLA$^+$ models live in `lux/formal/tla/`:

- `DAGEVM.tla` + `MC_DAGEVM.cfg` — state variables accepted, finalRank, mempool; actions Propose, Finalize; invariants `TypeOK`, `Safety`, `NoDoubleSpend`, `Serializability`, `ConflictsAlongEdges`; property `Liveness`.
- `DAGZChain.tla` + `MC_DAGZChain.cfg` — state variables accepted, spent, committed, finalRank; invariants `NoDoubleSpend`, `AntichainDisjoint`.

TLC model-checking runs against small instances ($|\mathsf{Keys}| = 3, |\mathsf{Vertices}| = 6$) and enumerates the full reachable state space in bounded time.

System	Exec parallelism	Consensus DAG?	Conflict source	State model
Ethereum	none	linear	–	EVM (account)
Aptos	Block-STM	linear	r/w sets	Move (resource)
Sui (Mysticeti)	object-type	DAG (Narwhal)	object IDs	Move (object)
Solana	account-lock	linear	static lockset	account
Monad	OCC + pipeline	linear	r/w sets (post-hoc)	EVM
Fuel	UTXO-parallel	linear	UTXO inputs	UTXO
Aleph Zero	none	DAG (AlephBFT)	–	substrate
Lux C-Chain (DAG-EVM)	Block-STM	**DAG**	**r/w sets**	EVM
Lux X-Chain	DAG-native	**DAG**	UTXO inputs	UTXO
Lux Z-Chain (DAG-Z)	parallel verify	**DAG**	nullifiers	ZK-UTXO
Lux Q-Chain	attest-parallel	**DAG+Quasar**	PQ stamps	attestation

Table 2: Lux versus contemporary parallel / DAG blockchains. Lux is the only production system in which (a) the execution engine's conflict graph is the consensus DAG and (b) this shape holds uniformly across EVM, UTXO, ZK-UTXO, and PQ attestation workloads.

7 Comparison with Contemporaries

The critical distinction: Sui's Narwhal DAG carries *transaction availability*, not the execution conflict graph. The DAG is collapsed to a total order by Bullshark/Mysticeti and then executed sequentially (or parallel by object-type, separately). Aptos' Block-STM operates inside a linear block: if block B_n is not yet finalised, no transaction of B_{n+1} can execute even if entirely independent. Monad stacks pipelining on linear consensus; DAG-level parallelism is not available. Solana's account-lock model is static (declared in the tx), not derived from execution footprints, and does not compose with DAG consensus.

Lux is the only production design in which the same conflict predicate drives intra-block execution (Block-STM) *and* inter-block finality (nebula DAG) — yielding a uniform model across all chains with a natural parallelism axis (EVM, DEX, UTXO, ZK, Oracle, Relay).

8 GPU-Native EVM Execution

The critical path of DAG-EVM is:

1. **Speculative Block-STM execution** of candidate vertex's txs.
2. **ecrecover** for every transaction in the batch.
3. **Conflict scan** against the frontier's bitmap union.
4. **keccak** for vertex ID and inclusion in the next level's Merkle-Patricia state root.

Through the v0.20.0 → v0.23.0 release cycle (luxcpp/evm tags `v0.22.0`, `v0.23.0`), the GPU stage stopped being a single ecrecover kernel and grew into a full opcode-level interpreter for both Apple Metal and NVIDIA CUDA, sharing one host-side dispatcher. The result is that every step above can run on the same GPU device.

8.1 Kernel Surface

The GPU EVM kernel is a single-source artefact in two dialects:

- `lib/evm/gpu/kernel/evm_kernel.metal` (1067 SLOC, Metal Shading Language 3.0).
- `lib/evm/gpu/cuda/evm_kernel.cu` (1713 SLOC, CUDA C++ for sm_80 / sm_90).

Both implement the Cancun-era opcode set including `KECCAK256`, the Cancun context family (`ORIGIN`/`GASPRICE`/`BLOCKHASH`/`CHAINID`/`BASEFEE`/`BLOBHASH`/`BLOBBASEFEE`), `MCOPY`, `TLOAD`/`TSTORE`, and `LOG0`–`LOG4`. ABI version 3 of the kernel adds a `BlockContext` buffer plus transient-storage and log buffers (Metal binds them at indices 8–12). Account-state opcodes (`BALANCE`, `EXTCODESIZE`, …) return safe defaults inside the kernel; live host data is provided through the bridge described in §8.3.

8.2 Precompile Dispatch

The precompile dispatcher routes addresses `0x01`–`0x11` through GPU code paths where it is profitable. `ECRECOVER` (`0x01`) executes on Metal and CUDA via `secp256k1_recover.cu`; the BLS12-381 (`0x0b`–`0x11`) EIP-2537 family is wired through the same dispatcher. The standalone unit target `evm-precompile-test` reports 19 precompile groups passing on the M1 Max reference hardware; the run is guarded by `available_backends()` so a host without a Metal device falls back to CPU silently.

8.3 EVMC Host Bridge for CALL/CREATE

The headline limitation of a pure-GPU EVM is that the kernel cannot make a runtime callback into an EVMC `Host` for nested `CALL`/`CREATE` frames; GPUs do not call back to CPU memory mid-warp. The v0.22.0 host bridge (`lib/evm/gpu/host/evmc_bridge.cpp`) sidesteps this by performing a static analysis on the entrypoint bytecode: it walks the basic-block graph, identifies every `CALL`/`CREATE` target whose address is a constant immediate, pre-resolves the destination code from the supplied `evmc::Host`, and packs the entire reachable code closure into a single GPU-side *code dictionary*. Dynamic-target calls (`CALL` where the address is computed at runtime) are rejected and the transaction falls back to evmone on CPU.

The accept/reject decisions and depth enforcement are exercised by `test/unittests/host_bridge_test.cpp`, with the seven scenarios listed in Table 3. The bridge enforces a maximum frame depth of 4; deeper chains route to CPU.

Scenario	**Outcome**
Pure leaf (no CALL/CREATE)	GPU accept
Static `CALL` with const target	GPU accept (pre-resolved)
Dynamic `CALL`	CPU fallback
Deterministic `CREATE`	GPU accept (deterministic addr)
4-deep static `CALL` chain	GPU accept
9-deep static `CALL` chain	CPU fallback (depth > MAX)
`DELEGATECALL` context	GPU accept; preserves caller

Table 3: Host-bridge accept/reject scenarios. All seven cases are checked byte-for-byte against the same transaction routed through evmone on CPU in `test/unittests/host_bridge_test.cpp`.

8.4 Unified Pipeline

A single Metal device drives EVM execution, consensus state hashing (`lib/evm/gpu/gpu_state_hasher.hpp`), and post-quantum signature verification (BLS12-381 + ML-DSA-65 hosted by `lib/evm/gpu/metal/bls_host.mm`). The pipelined run measured at v0.22.0 on M1 Max reports a 1.07× speedup vs serial (consensus + EVM + sig-verify) at 19 blocks/s sustained over 1000 blocks — the modest factor reflects the fact that each stage already saturates a different GPU resource (compute for EVM, memory bandwidth for keccak state hashing, BLS scalar-mult for sig-verify), so pipelining recovers only the overlap. Source: `lib/evm/gpu/test_unified_pipeline.cpp`.

8.5 Empirical Speedup, M1 Max

Two regimes need separate accounting: (a) the original ecrecover/keccak-dominated bottleneck that motivated the GPU port at v0.20.0, and (b) the bytecode-level kernel that came online at v0.22.0. Table 4 shows the ecrecover/keccak path is essentially solved (the original 32× ecrecover speedup carries over) and that the bytecode kernel itself is faster than evmone on per-opcode microkernels once the batch is large enough to amortise GPU dispatch.

Stage	**CPU evmone**	**GPU Metal**	**Speedup**
ecrecover (47K sigs)	1613 ms	50 ms	32.3×
keccak256 (47K hashes)	127 ms	18 ms	7.1×
Bytecode kernel (`evm-bench-kernel`, ADD-loop tx)			
GPU V1 (1 thread/tx)	—	252 M ops/s	1.62×
CPU evmone reference	155 M ops/s	—	1.00×
Real ADD-loop tx (50 iter, N=2048)			
GPU V1	—	25,327 TPS	—
		1.14 M EVM ops/s	—

Table 4: Measured GPU EVM speedups on Apple M1 Max, 32-core Metal GPU. ecrecover and keccak figures reproduce in `lib/evm/gpu/metal/bench_keccak.cpp`. Bytecode-kernel figures from `lib/evm/gpu/kernel/bench_kernel.cpp`; gas-match against CPU evmone is asserted on every run (the bench fails closed if a single tx's `gas_used` disagrees). The GPU kernel does not yet beat the Block-STM CPU-Parallel path on every microkernel — `results/speedup-with-buffer-cache-2026-02-28.txt` catalogs the $N \leq 2048$ regimes where CPU-Parallel still wins; that is the workload where a per-thread CPU evmone interpreter outpaces a one-tx-per-warp GPU launch. The bytecode kernel wins above this batch size and on workloads where ecrecover dominates.

GPU is auto-detected at runtime: if the dispatcher's `available_backends()` returns a non-CPU backend, the bytecode is routed through the GPU host launcher (`evm_kernel_host.mm` on Apple, `cuda/evm_kernel_host.cpp` on NVIDIA). When no GPU is present, both CPU paths (`CPU_Sequential` and `CPU_Parallel` via Block-STM) run unchanged. There is no user flag.

8.6 Limitations

- *Dynamic CALL targets fall back to CPU.* The host-bridge static analysis only resolves constant-immediate targets. Solidity code that loads a callee address from storage at runtime is not yet GPU-eligible. Path: extend the dictionary to include the closure of all addresses ever stored in dispatcher slots, with re-staging on dictionary miss. Not yet implemented.

- *`SELFDESTRUCT` is rejected.* The kernel returns `CallNotSupported` on `0xFF`; the dispatcher falls back to CPU. Acceptable today because Cancun deprecates the opcode for new accounts.
- *Account-state opcodes return defaults inside the kernel.* `BALANCE`, `EXTCODESIZE`, `EXTCODEHASH`, `EXTCODECOPY`, `SELFBALANCE` read from a host-supplied snapshot dictionary; addresses missing from the dictionary surface as zero. The host bridge populates the dictionary for every address it has pre-resolved; addresses outside that closure read zero. The parity test corpus catches this with the `state_*` vectors.
- $N \leq 2048$ *batches lose to CPU-Parallel.* GPU dispatch cost (host → device buffer staging + threadgroup launch) is in the 6 ms range on M1 Max for cold buffers, dropping to ∼4 ms once buffer cache is warm. CPU-Parallel Block-STM clears the same workload in ∼0.1 ms–4 ms. The crossover moves into the kernel's favour at $N \geq 4096$ (workload-dependent), or when the workload contains ecrecover/keccak which still dominate on CPU.

9 Plugging the GPU EVM into Quasar Consensus

The previous section established that a single Metal/CUDA kernel can execute a Cancun-era block end-to-end. This section answers a separate question: *where* in the Quasar consensus pipeline does that execution actually get invoked, against *which* interface, and at *what* batch sizes does the GPU win.

9.1 What Quasar Decomposes Into

The consensus repository (`github.com/luxfi/consensus` at SHA `5bfc6ac`) is a pure-protocol library: it contains no execution logic and no state model. It exposes two engine modes wired from the same sub-protocols:

- **Linear chain** (`protocol/nova`, driven by `engine/chain`) — Photon → Wave → Focus → Ray → Sink. Used by the C-Chain.
- **DAG** (`protocol/nebula`, driven by `engine/dag` and `protocol/field`) — Photon → Wave (per frontier vertex) → Flare (cert/skip) → Horizon (safe-prefix scan) → Field (commit). This is the path described in §8 for the DAG-EVM.

The components, with file references in `~/work/lux/consensus`:

- *DAG vertex production.* `protocol/field/field.go:30` defines `Proposer[V].Propose(ctx, parents) (V, error)`. The *networking* layer pushes a payload into the proposer; the proposer returns a vertex ID and ships it. No bytecode runs here — a vertex is just `(parents, author, round, payload bytes)`.
- *Wave-based ordering.* `protocol/wave/wave.go` (the `Wave[T]` struct) drives threshold voting per frontier vertex. The FPC selector (`protocol/wave/fpc/`) randomises the threshold $\theta \in [\theta_{\min}, \theta_{\max}]$ from a PRF seed to defeat adaptive adversaries. Wave alone returns `(preferOK, confOK)` per round; it does not commit.
- *FPC final commitment.* `protocol/field/field.go:107` `Driver[V].Tick` computes a *safe prefix* via `computeSafePrefix` (line 119), then calls `Committer[V].Commit(ctx, ordered)`. The committer's `Commit` call is the post-FPC hook: every vertex passed in is finalised with all its causal ancestors finalised.

- *Quasar certificate.* `protocol/quasar/types.go:40` `QuasarCert` carries the BLS aggregate, Ringtail share, and Z-Chain Groth16-rolled-up ML-DSA proof. The cert is produced *after* `Commit` returns; it certifies the safe prefix, not individual transactions.

9.2 Where Bytecode Execution Lives

The crucial point: *Quasar does not call into the EVM.* Quasar orders opaque vertex IDs. The EVM call lives one layer above Quasar, in the VM that hosts the chain.

Linear chain (Nova). `engine/chain/block/block.go:88--130` defines `ChainVM`, which the engine drives. The engine's call sequence (`engine/chain/engine.go:947--1019`, function `buildBlocksLocked`) is:

1. `vm.BuildBlock(ctx)` returns a `block.Block` (an interface, not bytes; line 94 of `block.go`).
2. *Engine releases its mutex* (line 976) and calls `vmBlock.Verify(ctx)` (line 977). **This is the EVM execution hook.** `Verify` runs the bytecode and returns the post-state. If `Verify` fails, the block is dropped and never enters consensus.
3. Engine adds the block to consensus (`t.consensus.AddBlock`, line 983) and self-votes.
4. On finalisation: `vmBlock.Accept(ctx)` (line 1000 for K=1, line 877 for the multi-validator finaliser `tryFinalizeBlock`, and line 924 for the engine's `DrainAccepted` loop).

So bytecode runs *during vertex production*, before consensus voting. `Accept` is a state-commitment hook — it persists what `Verify` already computed, it does not re-execute.

DAG mode (Nebula). The same split shifts one level: `Proposer.Propose` runs the bytecode for the proposed vertex's transactions and returns a vertex ID that already commits to the post-state root. `Field.Commit` then delivers an *ordered* list of vertex IDs whose post-state was computed at proposal time. The committer's job is to *apply* those state diffs against the now-canonical history — not to re-run the EVM.

This split is forced by the protocol shape: Quasar votes on vertex IDs, and a vertex's ID must commit to its post-state for the cert to be meaningful. We therefore execute speculatively at proposal, and apply deterministically at commit. The Block-STM scheduler in `cevm` (`lib/evm/gpu/parallel_engine.cpp`) provides the speculative re-execution-on-conflict that this requires.

9.3 The Interface Quasar Calls

The full `ChainVM` surface area Quasar's chain engine drives, from `engine/chain/block/block.go:88--130` (verbatim):

```
type ChainVM interface {
    Initialize(ctx context.Context, init Init) error
    BuildBlock(context.Context) (Block, error)
    ParseBlock(context.Context, []byte) (Block, error)
    GetBlock(context.Context, ids.ID) (Block, error)
    SetPreference(context.Context, ids.ID) error
    LastAccepted(context.Context) (ids.ID, error)
    GetBlockIDAtHeight(context.Context, uint64) (ids.ID, error)
    SetState(context.Context, uint32) error
```

```
    Shutdown(context.Context) error
    Version(context.Context) (string, error)
    Connected(context.Context, ids.NodeID, *version.Application) error
    Disconnected(context.Context, ids.NodeID) error
    HealthCheck(context.Context) (HealthCheckResult, error)
    NewHTTPHandler(context.Context) (http.Handler, error)
    WaitForEvent(context.Context) (Message, error)
}

type Block interface {
    ID() ids.ID
    Parent() ids.ID
    ParentID() ids.ID
    Height() uint64
    Timestamp() time.Time
    Status() uint8
    Verify(context.Context) error   // <-- bytecode here
    Accept(context.Context) error   // <-- commit state here
    Reject(context.Context) error
    Bytes() []byte
}
```

For the DAG path, `engine/dag/vertex/vertex.go:11--28` adds:

```
type Vertex interface {
    ID() ids.ID
    Bytes() []byte
    Height() uint64
    Epoch() uint32
    Parents() []ids.ID
    Txs() []ids.ID
    Status() choices.Status
    Verify(context.Context) error
    Accept(context.Context) error
    Reject(context.Context) error
}
type DAGVM interface {
    ParseVertex(context.Context, []byte) (Vertex, error)
    BuildVertex(context.Context) (Vertex, error)
}
```

Two hooks, both for the GPU EVM: `Verify` for execution, `Accept` for state commit.

9.4 Concrete Integration Glue

The `cevm` package (`~/work/lux/cevm/cevm.go` at SHA `4d652b2`) already exports the C ABI:

```
func ExecuteBlockV2(backend Backend, numThreads uint32,
    txs []Transaction) (*BlockResultV2, error)
```

The integration component is a Go struct that satisfies `block.Block` and routes `Verify` into `cevm.ExecuteBlockV2`. The plumbing is direct because the consensus library already keeps execution and ordering decoupled — the only state that flows from the cevm result into the consensus layer is the 32-byte state root (carried in `Block.Bytes()`) and the boolean `Verify` return.

```
// cevmBlock satisfies engine/chain/block.Block and engine/dag/vertex.Vertex.
type cevmBlock struct {
    id        ids.ID
    parent    ids.ID
    height    uint64
```

```
    ts        time.Time
    txs       []cevm.Transaction
    backend   cevm.Backend
    threads   uint32
    // populated by Verify, read by Accept:
    result    *cevm.BlockResultV2
    stateRoot [32]byte
    raw       []byte
    state     uint8 // choices.Processing / Accepted / Rejected
    parent2   *cevmBlock // for state diff application at Accept
    db        kvWriter
}

func (b *cevmBlock) ID() ids.ID         { return b.id }
func (b *cevmBlock) Parent() ids.ID     { return b.parent }
func (b *cevmBlock) ParentID() ids.ID   { return b.parent }
func (b *cevmBlock) Height() uint64     { return b.height }
func (b *cevmBlock) Timestamp() time.Time { return b.ts }
func (b *cevmBlock) Status() uint8      { return b.state }
func (b *cevmBlock) Bytes() []byte      { return b.raw }

// Verify is the GPU EVM hook. Called by engine/chain/engine.go:977.
func (b *cevmBlock) Verify(ctx context.Context) error {
    if b.result != nil {
        return nil // memoised
    }
    res, err := cevm.ExecuteBlockV2(b.backend, b.threads, b.txs)
    if err != nil {
        return fmt.Errorf("cevm verify height=%d: %w", b.height, err)
    }
    // Cross-check vs the proposer's claimed root in raw block bytes.
    if !bytes.Equal(res.StateRoot[:], b.expectedRoot()) {
        return errStateRootMismatch
    }
    // Reject any tx with TxError; TxRevert is a valid in-protocol failure.
    for i, s := range res.Status {
        if s == cevm.TxError {
            return fmt.Errorf("tx %d returned TxError", i)
        }
    }
    b.result = res
    copy(b.stateRoot[:], res.StateRoot[:])
    return nil
}

// Accept is the state-commit hook. Called by engine/chain/engine.go:877,
// engine.go:924 (DrainAccepted), or engine.go:1000 (K=1 fast path).
func (b *cevmBlock) Accept(ctx context.Context) error {
    if b.result == nil {
        return errVerifyNotRun
    }
    // Persist the diff produced by Verify. The cevm bridge surfaces gas
    // and status; the underlying state diff is pulled from the host
    // bridge's account-state dictionary.
    if err := b.db.applyDiff(b.result, b.stateRoot); err != nil {
        return err
    }
    b.state = choicesAccepted
    return nil
```

```
}

func (b *cevmBlock) Reject(ctx context.Context) error {
    b.state = choicesRejected
    b.result = nil // free GPU result
    return nil
}
```

The VM-level `ChainVM.BuildBlock` constructs a `cevmBlock` from the mempool, picks the backend (`cevm.AutoDetect()` at boot, `cevm.Health()` verified), and returns it. The engine then drives `Verify`, votes, and on quorum calls `Accept`. No code in `consensus/` changes; the GPU EVM is a pure plug into `block.ChainVM`.

9.5 Where the Two Parallelisms Meet

Quasar can finalise multiple DAG vertices concurrently — specifically, any antichain of the conflict-free DAG (§??). The GPU EVM can execute thousands of transactions concurrently within one launch. These are *orthogonal* dimensions:

Source	Granularity	Limit
Quasar DAG width	vertices per round	antichain size of DAG
GPU EVM batch size	txs per dispatch	GPU SIMD lane count

The dimension that dominates is the one with the lower throughput floor. On M1 Max the GPU launch cost is 6 ms cold, 4 ms warm (§8.5); the per-vertex proposer cost on the consensus side is dominated by the wave round-trip (250 ms default `RoundTO`, `protocol/field/field.go:67`). *Consensus dominates by ∼40×.* The GPU has plenty of slack to amortise launch cost across a vertex's transactions, but it cannot make the wave round shorter.

Pipelining helps when the GPU launch on vertex V_n overlaps with the wave round on V_{n-1}. In the linear chain mode, the engine signals `t.signalPipeline()` after every accept (`engine.go:894`), so a build/verify can begin while the previous block is still being gossiped. Pipelining hurts when the proposer must wait for the GPU launch to drain before issuing a fresh `Verify` — the cevm `ExecuteBlockV2` is currently synchronous (the C bridge blocks on `[commandBuffer waitUntilCompleted]`). Async dispatch is listed as an open item in `lib/evm/gpu/PERF.md` (priority 2).

9.6 Saturation Math

The required regime, given the cevm dispatch crossover plot (`lib/evm/gpu/PERF.md`, "Crossover analysis"):

$$
\begin{aligned}
T_{\text{launch}}^{\text{cold}} &= 6\,\text{ms} \\
T_{\text{launch}}^{\text{warm}} &= 4\,\text{ms} \\
T_{\text{op}}^{\text{GPU}} &= \frac{1}{339\text{M ops/s}} \approx 2.95\,\text{ns/op} \\
T_{\text{op}}^{\text{CPU}} &= \frac{1}{159\text{M ops/s}} \approx 6.29\,\text{ns/op}
\end{aligned}
$$

Break-even (warm) for a generic workload of W opcodes per dispatch:

$$T_{\text{launch}}^{\text{warm}} + W \cdot T_{\text{op}}^{\text{GPU}} < W \cdot T_{\text{op}}^{\text{CPU}} \quad \Rightarrow \quad W > \frac{T_{\text{launch}}^{\text{warm}}}{T_{\text{op}}^{\text{CPU}} - T_{\text{op}}^{\text{GPU}}} \approx \frac{4\,\text{ms}}{3.34\,\text{ns}} \approx 1.2 \times 10^6 \text{ opcodes.}$$

So a single dispatch needs ∼1.2M opcodes for the GPU to win strictly on bytecode. The 100 K txs/sec target translates as follows. A vertex of 100 txs at 50 EVM ops/tx delivers 5000 ops per dispatch — well *below* the break-even. A vertex of 100 txs at 600 ops/tx (loop-heavy contracts) delivers 60,000 ops — still below. The break-even at 1.2M ops requires either:

1. *Fat workload per dispatch:* 100 txs × 12,000 ops/tx (heavy contract — DEX swap, AMM compound), or
2. *Bigger batch:* 12,000 txs at 100 ops/tx, or
3. *Pipelined dispatch* that amortises launch across multiple blocks.

The 100 K txs/sec (1000 vertices × 100 txs) *aggregate* is more than enough — if 1000 vertices/sec is emitted from a wave of 5–10 parallel proposers, each proposer dispatches to a separate cevm host (`cevm.ExecuteBlockV2` is goroutine-safe per `cevm.go:17`, with thread-local kernel hosts). Aggregating across proposers gives 200–1000 txs per dispatch on average, still below break-even at the 50-op-per-tx floor.

The ecrecover and keccak paths are different. Their break-even sits much lower because the per-op CPU cost is high (34 µs/sig for ecrecover — Table 4). At 47,000 signatures per dispatch the GPU runs in 50 ms versus the CPU's 1613 ms. A 100-tx block with two signatures per tx (200 sigs) is below crypto break-even too; a 10,000-tx block (or 50 blocks of 200 sigs each batched together) is firmly above it. *In production, signature verification is the path that justifies the GPU; opcode execution wins only on fat workloads or via pipelining.*

9.7 Limitations

- *No host callback at vertex boundary.* `cevm.ExecuteBlockV2` is a single C call that returns when the full batch finishes. The GPU kernel cannot ask Quasar mid-dispatch "did this vertex finalise yet?" because there is no host-callback primitive in the bridge. Consequence: the EVM cannot observe consensus state changes (e.g., a finalised cross-shard read) within a single `Verify`. The host bridge handles `CALL`/`CREATE` only via static analysis on the entry bytecode (§8.3); cross-vertex callbacks would require either re-issuing a fresh dispatch or a CPU fallback path.
- *Synchronous dispatch.* `[commandBuffer waitUntilCompleted]` blocks the calling goroutine. While the consensus engine has a per-block goroutine pool (`engine.go buildBlocksLocked`), a backed-up GPU draining a fat dispatch will pile up subsequent `Verify` calls. Async dispatch (priority 2 in PERF.md) is required for true pipelining across vertices.
- *Account-state opcodes return defaults.* `BALANCE`, `EXTCODESIZE`, `EXTCODEHASH`, `EXTCODECOPY`, `SELFBALANCE` read a host-supplied snapshot dictionary. For a vertex that touches an address outside the static-analysis closure, the kernel reads zero. The integration glue (`cevmBlock.Verify`) does not detect this and would accept a wrong-answer block; the parity-test corpus (§10) is the only catch. A *stricter* integration would re-run on CPU when any opcode misses the dictionary — not yet wired.

- *Dynamic-target `CALL` forces CPU fallback.* Per §8.3, a runtime-computed callee address routes the entire transaction to evmone on CPU. This is correct but breaks the GPU-batch invariant: a block of 100 txs in which one tx has a dynamic CALL splits into one CPU run plus a 99-tx GPU run, doubling the total wall-clock for that block.
- *Quasar does not vote on state roots, only vertex IDs.* A vertex's payload bytes commit to the post-state root, but Quasar itself never re-checks that root. If two honest validators execute the same vertex on different cevm backends (e.g., one Metal, one CPU-Parallel) and produce different roots due to a parity bug, only the four-way parity test corpus (`test/parity_test.cpp`) catches it before deployment. Run-time disagreement would surface as a vote split, not a halt — so divergent backends would silently lose finality on the affected branch. The fix path is exactly the parity corpus discipline: every backend must be byte-equivalent on every consensus-critical vector.
- *No state-streaming for huge dispatches.* The cevm result is materialised entirely in host RAM (the V2 ABI returns a single `state_root[32]` plus per-tx arrays). A 100 K-tx dispatch that needed to stream state diffs to a database during execution cannot be expressed in the current ABI. Block-STM provides the speculation; persistence is bulk-applied at `Accept`. For bytecode that mutates millions of slots, this means the GPU result sits in RAM until the consensus quorum lands, which is acceptable at today's chain TPS but will need a streaming path before a single GPU dispatch routinely exceeds available host RAM.

Net. The integration is small — a `block.Block` adapter, one C call per `Verify`, no consensus changes — because the consensus library is intentionally execution-free. The hard work is on the GPU side (parity, host bridge, V2 kernel for SIMD-cooperative dispatch). The right way to read this section is that the seam is already correct; what remains is closing the break-even gap so that GPU dispatch wins at vertex sizes the network actually emits.

10 Four-Way Parity as a Production Argument

The claim of consensus-equivalent execution across four backends is not a slogan; it is a single test driver that compiles, links, and runs all four in the same process.

10.1 The Four Backends

`evm::gpu::Backend` enumerates four routes, each invoked through a shared dispatcher entrypoint `execute_block(config, txs, nullptr)`:

1. `CPU_Sequential` — the kernel CPU interpreter (`lib/evm/gpu/kernel/evm_interpreter.hpp`, header-only), single-threaded, opcode-for-opcode reflection of the Metal kernel. This is the *spec witness*: it shares the source of truth with the GPU kernels.
2. `CPU_Parallel` — the same interpreter run by the Block-STM scheduler with N worker threads (`lib/evm/gpu/parallel_engine.cpp`).
3. `GPU_Metal` — the Apple Metal kernel (`evm_kernel.metal`), one threadgroup per transaction, dispatched by `lib/evm/gpu/kernel/evm_kernel_host.mm`.
4. `GPU_CUDA` — the NVIDIA CUDA kernel (`lib/evm/gpu/cuda/evm_kernel.cu`), one warp per transaction, dispatched by `lib/evm/gpu/cuda/evm_kernel_host.cpp`.

The CUDA arm is gated on the build define `EVM_CUDA`. On macOS the define is absent so the CUDA assertions compile out and only Apple GPU remains in the loop; on Linux/CUDA CI (the AWS EKS workflow described in §11.4) the define flips on and all four backends are checked.

10.2 Vector Corpus

The corpus is built once per process startup (`test/unittests/parity_test.cpp`, lines 92–573). It contains 133 bytecode vectors hand-written to exercise:

- *Arithmetic*: `ADD`, `SUB`, `MUL`, `DIV`, `MOD`, `ADDMOD`, `MULMOD`, `EXP`, signed variants;
- *Comparison* and *bitwise*: signed/unsigned `LT`/`GT`/`SLT`/`SGT`, `ISZERO`, `AND`/`OR`/`XOR`/`NOT`, `BYTE`/`SHL`/`SHR`/`SAR`;
- *Hashing*: `KECCAK256` on empty / `abc` / 32-byte-zero / 256-byte-zero inputs;
- *Context*: `ADDRESS`, `ORIGIN`, `CALLER`, `CALLVALUE`, `CHAINID`, `GASPRICE`, `COINBASE`, `TIMESTAMP`, `NUMBER`, `GASLIMIT`, `BASEFEE`, `BLOBHASH`, `BLOBBASEFEE`, `BLOCKHASH`;
- *Calldata*, *code*, *memory* (`MSTORE`/`MLOAD`/`MSTORE8`/`MSIZE`/`MCOPY`);
- *Storage* (`SSTORE`/`SLOAD`, `TSTORE`/`TLOAD`, zero-load);
- *Control flow* (`JUMP`, `JUMPI`, `PC`, `GAS`, `STOP`, `INVALID`, undefined opcode, bad jump);
- *Stack*: `PUSH0` through `PUSH32`, `DUP1`–`DUP16`, `SWAP1`–`SWAP16`, `POP`;
- *Logging* (`LOG0`–`LOG4`);
- *Returndata* (`RETURN`/`REVERT`/empty/word);
- *Account state* (default-zero through the kernel);
- *CALL family* expected to return `CallNotSupported` (`CALL`, `CALLCODE`, `DELEGATECALL`, `STATICCALL`, `CREATE`, `CREATE2`, `SELFDESTRUCT`);
- *Out-of-gas*.

Each vector carries an `Expectation` tag — `Agree`, `GasOnly`, or `KernelCpuMissing` — that tells the test which divergences are tolerated. `Agree` demands bit-exact match on `(gas_used, status, output)` across all four backends. `GasOnly` permits gas-accounting drift (typically the cost of `MSTORE` memory expansion in different revisions) but still requires status and output to match. `KernelCpuMissing` marks vectors where the kernel CPU interpreter has not yet implemented an opcode that the GPU kernels do; these vectors expect CPU `Error` and GPU success — catching the case where a new opcode lands on GPU before the CPU spec witness catches up.

10.3 The Test

`TEST(Parity, AllBackendsAgreeOnEveryVector)` loops the corpus once, runs each vector through all four backends, and counts the `(Agree ok, GasOnly ok, Missing ok)` totals. Any divergence under the active expectation registers as `ADD_FAILURE`; the test asserts at the end that the failure count is zero. The test is a single GoogleTest case so it can run inside `ctest -j` alongside the other unit suites.

The output prints, on every CI run:

```
[parity] runs:  CPU_Sequential=N CPU_Parallel=N GPU_Metal=N [GPU_CUDA=N]
[parity] Agree:            <ok> ok, <fail> fail
[parity] GasOnly:          <ok> ok, <fail> fail
[parity] KernelCpuMissing: <ok> ok, <fail> fail
[parity] total <pass> / <total> vectors meet expectations
```

10.4 Why This Is Not a Smoke Test

Three properties make four-way parity a production-grade argument rather than a developer-confidence test.

(P1) Independent implementations of the same spec. The kernel CPU interpreter and the Metal kernel share a directory and review cycle but no opcode-level code. CUDA was ported from Metal in a separate branch by line-by-line translation (§11) and produces an independent control-flow graph: the Metal version uses `threadgroup` reductions for stack ops, the CUDA version uses warp shuffles. Block-STM adds a fourth code path because the scheduler's read/write tracking is a different layer of the system. A bit-exact result across all four is therefore evidence that the EVM specification was internalised, not that one implementation was copy-pasted.

(P2) Status, gas, and output checked exactly. The default `Expectation::Agree` requires equality on three independent quantities. Status is a discrete enum; gas is a 64-bit integer; output is a byte vector of any length. There are 13 vectors marked `GasOnly` and a small set marked `KernelCpuMissing` — each tagged exception is a documented divergence with a specific algebraic cause, not a mass-pardon. The test writes a `[parity]` promotion hint to `stdout` when a `GasOnly` vector starts agreeing, forcing the engineer either to upgrade the tag or explain the regression.

(P3) The four backends are co-deployed. The same dispatcher `execute_block(config, txs, nullptr)` is what production callers use. There is no separate "test mode" path. A consensus-relevant bug that crept into the Metal kernel would be caught by the CUDA assertion; a CUDA-only bug would fail the Metal assertion; a kernel-CPU-only bug would fail either GPU. The cross-checking is symmetric and runs on every PR through CI.

10.5 What the Coverage Numbers Add

Source-based LLVM coverage is built into the same target set (`lib/evm/COVERAGE.md`). The parity-driver run at v0.23.0 reports **90.87% line coverage and 100% function coverage** on the kernel CPU interpreter (`evm_interpreter.hpp`). The 9.13% of unexecuted lines are precisely the unimplemented `CALL`/`CREATE` arms (which the test catches under `Expectation::Agree` for `CallNotSupported`) and the dead branches behind compile-time `assert`s. The combination — four backends agreeing across 133 hand-crafted vectors plus 100% function coverage on the spec witness — means we have a counter-example finder if any backend silently diverges from the others on an opcode the corpus already touches.

10.6 Limitations

- *Coverage on `.metal` and `.cu` sources is structural only.* LLVM source-based coverage instruments the kernel-CPU interpreter (which is a `.hpp`) and the host launchers (`.mm`/`.cpp`). Metal AIR and CUDA PTX are outside LLVM's coverage scope (Metal compiles to AIR, nvcc emits PTX without coverage-mapped IR). Coverage on the GPU code itself is indirect: every vector that executes through the GPU dispatcher necessarily executes the corresponding device path. The four-way parity test is the corroboration.

- *The corpus is per-opcode, not stateful.* Each vector runs in its own fresh state. A consensus bug that requires multiple transactions to manifest (a Block-STM hazard, an SSTORE refund accumulator inconsistency across txs) is not currently in the corpus. The Block-STM scheduler has its own unit suite (`lib/evm/gpu/parallel_engine.cpp` drives `evm-test-modes`) but it does not cross-validate against the other three backends. Path: extend `ParityVector` to a `ParitySequence` type carrying multiple `Transaction`s and a state-checkpoint diff. Not yet implemented.
- *No fuzzing layer.* The corpus is hand-written and grows by inspection. A differential fuzzer that emits random bytecode and checks four-way agreement would close more of the long tail. `test/fuzzer/` hosts the existing evmone fuzzer; wiring it through the GPU dispatcher is on the v0.24 roadmap.

11 CUDA Port: Methodology and Inherited Bugs

The CUDA port of the GPU EVM kernel arrived as a sequence of three commits on the `feat/cuda-evm-kernel` branch (`b5ce473`, `16db493`, `2e3022a`, all merged Feb 2026), bringing the CUDA kernel from a stub at v0.21.0 to opcode-complete at v0.22.0 (`lib/evm/gpu/cuda/evm_kernel.cu`, 1713 SLOC).

11.1 Methodology

The Metal kernel was the source of truth. The port proceeded in four passes:

1. *Type carry-over.* The `uint256`, `pair64`, and storage-pool descriptor types defined in `kernel/uint256_gpu.hp` are shared between Metal and CUDA; only the address-space qualifiers differ (`device` vs `__device__`). The CUDA dialect uses `__host__ __device__` where the Metal version uses plain `static inline`.
2. *Bit-by-bit opcode translation.* Each opcode handler from Metal's `switch` block was translated into the corresponding CUDA case, preserving stack semantics, gas accounting, and emit-on-error behaviour. The Metal-specific bits — `threadgroup_barrier`, `simdgroup_*` — were replaced with CUDA equivalents (`__syncthreads`, `__shfl_sync`).
3. *Host launcher parity.* The Apple host (`evm_kernel_host.mm`) and the CUDA host (`cuda/evm_kernel_host.c` implement the same `HostTransaction` ABI and dispatch through the same `gpu_dispatch.hpp` interface. The dispatcher's backend probe selects whichever device is present at runtime.
4. *Cross-validation.* The fourth backend was wired into the parity corpus (§10) under the `EVM_CUDA` define. The first run flushed the bugs catalogued below.

11.2 Three Inherited Bugs

The translation surfaced three consensus-relevant bugs that were already latent in the Metal kernel but had not yet been caught. The CUDA port acted as a differential oracle: identical bytecode through two implementations exposed where each was wrong against the EVM spec, not just against each other.

Bug 1: `u256_mulmod` carry-loss in upper limbs. The original 256-bit modular-multiply followed the textbook schoolbook loop, accumulating a 4×4 partial-product matrix into an 8-limb scratch buffer:

```
gpu_u64 r8[8] = {0, ..., 0};
for (uint i = 0; i < 4; ++i) {
    gpu_u64 carry = 0;
    for (uint j = 0; j < 4; ++j) {
        pair64 p = mul_wide(a.w[i], b.w[j]);
        s = r8[i+j] + p.lo;
        c = (s < r8[i+j]) ? 1 : 0;
        s += carry;
        c += (s < carry) ? 1 : 0;
        r8[i+j] = s;
        carry = p.hi + c;
    }
    if (i + 4 < 8) r8[i+4] += carry;  // BUG: i=3 lost
}
```

The guard `if (i+4 < 8)` is false for the last outer iteration (`i = 3`), so the carry leaving `a.w[3] * b.w[3]` was silently dropped. Even when the guard fired ($\texttt{i} \leq 2$), the naked `r8[i+4] += carry` could itself wrap around, and the secondary carry was never propagated to `r8[i+5]`.

The algebraic shape of the fix is to propagate the outer carry through *all* higher limbs:

$$\forall k \geq i+4: \quad (r'_k, c') := r_k + c \text{ (with } c' = \mathbf{1}[r_k + c < r_k]), \quad r_k \leftarrow r'_k,\ c \leftarrow c'.$$

In code (`evm_kernel.metal`, lines 204–239):

```
uint k = i + 4;
while (carry != 0 && k < 8) {
    gpu_u64 s = r8[k] + carry;
    gpu_u64 c = (s < r8[k]) ? 1UL : 0UL;
    r8[k] = s;
    carry = c;
    ++k;
}
```

The fix is identical in CUDA. The bug fires whenever $\lfloor a \cdot b / 2^{448} \rfloor > 0$, i.e. whenever the high limb of the product is non-zero, which is every input pair where both operands exceed 2^{192}. Real-world impact: any contract that uses `MULMOD` with values near group order in `secp256k1` or `bn254` (i.e. every contract that does on-chain elliptic-curve work) would see incorrect results. The bug was flagged `HIGH` in the internal red-team review before the parity corpus reproduced it.

Bug 2: Unrecognized opcodes did not consume all gas. The EVM spec mandates that an unrecognized opcode — not just the `0xFE INVALID` canonical reserved opcode, but any byte that is not a defined instruction — consumes all remaining gas of the current frame. The pre-fix kernel fell through to a generic `ERR()` macro that emitted `Error` status with the gas counter at its current value, which is wrong: the consensus-correct behaviour is to set `gas_used = gas_start` (the entire gas budget).

The fix is the new `ERRA()` macro on Metal kernel line 1059 (`ERR-all-gas`) which emits `(status=Error, gas=gas_start, output={})`. The `0xFE` canonical `INVALID` arm at line 1035 was already correct; the fix routes every unhandled opcode to the same path.

Bug 3: `MSTORE/MLOAD` offset+32 overflow. `MSTORE` (and `MLOAD`) take a 256-bit memory offset on the stack. The kernel implementation extracted the low 64 bits and performed the bounds check as

$$\mathsf{ok} := (\mathit{offset}_{\mathrm{low64}} + 32) \leq \mathsf{MAX\_MEM}.$$

For offsets near $2^{64} - 32$ (which is well below the EVM's $2^{256} - 1$ stack-representable range, but above the kernel's tx memory cap), the addition wrapped around and the comparison spuriously succeeded, allowing memory writes past the per-tx memory ceiling and corrupting the next transaction's slab in `mem_pool`. The fix (`evm_kernel.metal` lines 426–434) explicitly checks the high three limbs of the 256-bit offset, then the low 64 bits, then the addition for wraparound:

```
static inline bool offset_in_bounds(uint256 v, ulong extra) {
    if (v.w[1] | v.w[2] | v.w[3]) return false;
    if (v.w[0] > MAX_MEMORY_PER_TX) return false;
    ulong sum = v.w[0] + extra;
    if (sum < v.w[0])              return false;
    if (sum > MAX_MEMORY_PER_TX)   return false;
    return true;
}
```

The same algebraic check ports verbatim to CUDA.

11.3 What the LLVM Coverage Tells Us

LLVM source-based coverage on the kernel CPU interpreter (`evm_interpreter.hpp`, the spec witness) reports:

- **Line coverage**: 90.87%
- **Function coverage**: 100%
- **Region coverage**: ∼85% (target met)

The 9.13% of uncovered lines fall into three categories:

1. Unreachable arms behind `static_assert` guards (compile-time excluded but counted by the instrumenter);
2. The `CALL`/`CREATE` branches that intentionally return `CallNotSupported` — the parity test exercises these under the `KernelCpuMissing` expectation, but the line immediately following the early return is dead by design;
3. A handful of out-of-gas branches in `u256_exp` and `u256_mulmod` that fire only for inputs the corpus does not yet construct (extremely large bases with extremely large exponents that hit the `GAS_EXP_BYTE` multiplier ceiling).

The **100% function coverage** is the load-bearing claim: every opcode handler the kernel implements is entered by at least one parity vector. Combined with four-way agreement on bit-exact gas/status/output, this gives us a closed system: a divergence between any backend and any other backend is detectable by an existing test, on every commit.

What coverage does *not* certify is the path through Metal's AIR or CUDA's PTX after compilation. Metal compiles to AIR, which LLVM coverage cannot instrument; nvcc emits PTX without coverage-mapped IR. Coverage on the device kernels is therefore *indirect*: every parity vector that executes through the GPU dispatcher reaches the corresponding opcode handler in the device shader, and a divergence in that path manifests as an assertion failure in the parity test, not as a coverage gap. The four-way parity is the certification, not the coverage number.

11.4 CI Path: AWS EKS GPU Karpenter

The CUDA arm cannot run on the Apple-only developer workstation, so it runs on AWS EKS through GitHub-hosted Actions Runner Controller. `lux-gpu-ci/helm/karpenter-nodepool.yaml` declares an `EC2NodeClass` that provisions `g4dn.xlarge` (NVIDIA T4) spot instances on demand, with fallback to `g5.xlarge` (NVIDIA A10G) when T4 capacity is unavailable, scaling to zero 30s after the runner pod terminates. The runner-scaleset configuration ships the NVIDIA device plugin DaemonSet via Amazon Linux 2023 AMIs.

The CI workflow (`.github/workflows/cuda.yml` in the `luxcpp/evm` repository) runs nightly and on every PR that touches `lib/evm/gpu/cuda/**`. The job:

1. Configures CMake with `-DEVMONE_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES="80;90"`;
2. Builds `evm-cuda` (the device library) and `evm-gpu` (the dispatcher with `EVM_CUDA` defined);
3. Verifies the CUDA symbols are present in the static archive (`nm libevm-cuda.a`);
4. Runs `evm-bench-kernel` as a smoke check;
5. In a dependent job, builds the `cevm` Go bindings against the freshly-compiled CUDA libraries and runs the Go test suite under `CGO_ENABLED=1`.

A nightly H100 self-hosted runner (`labels: self-hosted, gpu, cuda, h100`) runs the same workflow on `sm_90` hardware to validate the warp-shuffle paths added at v0.22.0; the EKS T4 runners cover `sm_80` and `sm_86`.

11.5 Limitations

- *No CUDA Block-STM yet.* The CUDA path implements single-tx-per-warp execution; the multi-version memory and re-execution scheduler in `block_stm.cu` compiles and links but is not yet wired into the dispatcher's `CPU_Parallel` equivalent on CUDA. Workloads above the M1 Max crossover therefore take the CPU Block-STM path on Linux/CUDA hosts; this is correct but leaves performance on the table. Path: complete the `BlockStmGpu` CUDA host launcher (`block_stm_host.cpp`) and fold it under `Backend::GPU_CUDA_Parallel`. Tracked.
- *Warp divergence on small batches.* A single tx whose bytecode contains long branchless arithmetic (e.g. a Solidity library doing 256-bit multiply chains) keeps all 32 threads of the warp busy; a tx with heavy data-dependent branching wastes warp lanes. The parity corpus does not yet quantify this. Path: instrument the kernel with `__activemask()` sampling and add a divergence-cost column to the bench output. Open.

- *No pre-Ampere coverage.* The CMake configuration targets `sm_80` (Ampere) and `sm_90` (Hopper); pre-Ampere GPUs are excluded because the kernel uses Ampere-only intrinsics in the keccak inner loop. Note: AWS `g4dn.xlarge` carries the T4 (`sm_75`, Turing) which falls below this threshold — the EKS Karpenter pool above is configured to fall back to `g5.xlarge` (A10G, `sm_86`) when T4 instances are incompatible with the build. The nightly H100 self-hosted runner (`sm_90`) is the canonical production validation target.

12 Empirical Results

This section consolidates the v0.23.0 measurements that other sections reference into a single table organised by backend and workload. Hardware: Apple M1 Max (32-core Metal GPU, 10-core CPU, 64 GB unified memory), macOS 14.5, clang/llvm 17. CUDA figures are produced by the nightly H100 runner (§11.4); the EKS T4/A10G runners cover build correctness rather than headline performance.

12.1 What Each Number Measures

- *TPS (transactions per second)* — end-to-end throughput including dispatch overhead. Measured on real EVM bytecode, not microbenchmarks.
- *Mgas/s* — aggregate gas processed per second. The EVM spec attaches a fixed gas cost to every opcode, so this number is the throughput a block builder cares about.
- *M ops/s* — raw opcode dispatch rate from `evm-bench-kernel`. Computed by counting bytecode opcodes actually executed (not gas-weighted) per loop iteration. Used primarily to compare GPU-V1 vs CPU evmone on the same compute-bound workload.

12.2 Headline Table

12.3 Where the GPU Loses

The bytecode kernel does not yet beat CPU_Parallel on every microkernel batch size. The detailed sweep (`lib/evm/gpu/results/speedup-with-buffer-cache-2026-02-28.txt`) shows GPU/CPU_Parallel ratios of 0.01×–0.23× across sweep cells $N \in \{32, 128, 512, 2048\}$ and 19 opcode workloads. The pattern is consistent: at $N \leq 512$, dispatch overhead dominates (∼6 ms–12 ms of cold-buffer cost vs ∼0.1 ms–1.2 ms for the CPU side). The GPU wins on:

- Workloads where `ecrecover` or large-input `KECCAK256` dominate (i.e. real blocks containing many transactions, not microkernels);
- Compute-heavy bytecode at $N \geq 4096$ where the 6 ms dispatch amortises over many transactions;
- Pipelined workloads where consensus state-hashing and sig-verify share the same Metal device with EVM execution.

The 1.62× headline figure on `evm-bench-kernel` is representative: pure-compute ADD-loop bytecode at N=2048 with all ecrecover paths excluded, comparing GPU-V1 (one tx per thread) to CPU evmone single-threaded. The same workload through CPU-Parallel does better in absolute terms but is not the relevant comparison for an ecrecover-dominated production block.

Workload	Backend	Throughput	Notes	Source
Real EVM bytecode (50-iter ADD loop, 13 ops/iter)				
N=2048 txs	GPU_Metal V1	25,327 TPS	1.14 M EVM ops/s	`bench_kernel.cpp`
N=2048 txs	GPU_Metal V1	1.14 M ops/s	gas-match vs CPU	`bench_kernel.cpp`
N=2048 txs	CPU evmone	155 M ops/s	single thread	`bench_kernel.cpp`
N=2048 txs	GPU_Metal V1	252 M ops/s	1.62× vs CPU	`bench_kernel.cpp`
Block-STM ETH transfers (21000 gas/tx, 10K txs/block)				
10K transfers	CPU_Sequential	534 Ggas/s	evmone, 1 thread	`bench_block.cpp`
10K transfers	CPU_Parallel (4T)	—	4-worker Block-STM	`bench_block.cpp`
10K transfers	CPU_Parallel (10T)	—	saturated cores	`bench_block.cpp`
Cryptographic critical path				
47K ecrecover	CPU	1613 ms	evmone batch	`bench_keccak.cpp`
47K ecrecover	GPU_Metal	50 ms	32.3×	`bench_keccak.cpp`
47K keccak256	CPU	127 ms	evmone keccak	`bench_keccak.cpp`
47K keccak256	GPU_Metal	18 ms	7.1×	`bench_keccak.cpp`
Unified pipeline (EVM + state hash + sig-verify)				
1000 blocks	Serial Metal	20 Hz	one stage at a time	`test_unified_pipeline.cpp`
1000 blocks	Pipelined Metal	19 Hz	1.07× overlap	`test_unified_pipeline.cpp`
Parity & correctness				
133 vectors	4 backends	100% pass	gas/status/output bit-exact	`parity_test.cpp`
147 vectors	GPU_Metal	100% pass	per-opcode coverage	`test_opcodes.cpp`
19 groups	GPU dispatch	100% pass	0x01–0x11 precompiles	`precompile_test.cpp`
7 scenarios	EVMC bridge	100% pass	CALL/CREATE pre-resolve	`host_bridge_test.cpp`

Table 5: Measured GPU EVM performance and correctness. All performance numbers are minimum-of-three runs to filter Metal device-coalescing warmup; correctness numbers are exact pass counts on the named test binaries. Source columns reference paths under `lib/evm/gpu/` and `test/unittests/` in the `luxcpp/evm` repository at tag `v0.23.0`.

12.4 Open Headline Numbers (v0.24+)

- *H100 baseline.* The H100 self-hosted runner exists but the headline table above is M1 Max only; H100 ops/s and TPS for the same bench are pending the v0.24 publication of the `cuda-bench-results` artefact.
- *End-to-end real-block TPS.* The numbers above are microbenchmarks. A measurement that ingests a real Cancun-era block from mainnet (with mixed precompile + storage + CALL workload) and reports wall-clock through the full GPU dispatcher is the next publication target.
- *CUDA Block-STM speedup.* CPU_Parallel exists; GPU_CUDA runs single-tx-per-warp. Pipelining the multi-version memory scheduler onto CUDA gives the obvious next $\sim 4\times$ on Block-STM-bound workloads, but the integration is incomplete (§11 limitations).

13 PQZ Cross-Chain Parallelism

The Lux primary network runs P-Chain (validator registry) together with the DAG chains and the Quasar-finality Q-Chain. We collectively call the operational model **PQZ**: Primary validators,

Quasar certificates, and the ZK overlay. The question: does running all chains concurrently on the same validator set compromise per-chain safety?

The answer is *no*, by a straightforward independence argument formalised in `Consensus/PQZParallel.lean`. The key observations:

1. **Type-level chain scoping**: vertex types in Go are package-parameterised (`zkvm.Vertex`, `dexvm.DexVertex`, etc.); a cross-chain `Conflicts` call is not expressible.
2. **Quasar certificates are chain-local**: each antichain Quasar stamp binds a chain-ID and rank; stamps from different chains are independently verifiable.
3. **Validator signing is re-entrant**: a validator signing on k chains performs k independent BLS + ML-DSA + Ringtail operations whose cost scales linearly in k. No chain blocks another.
4. **Shared validator set, shared network**: the only cross-chain coupling is bandwidth; GPU batch verify linearises signature cost across all chains on the same hardware.

We measured this empirically. The 6 DAG-capable VMs (Z, D, A, O, R, S) are driven by 17 unit tests running concurrently in `go test -race`; no data races are reported. The `evmgpu/dag` package adds another 17 tests (`TestStorageKeySetDisjoint` through `TestVertexStore`) with GPU and CPU build-tag variants. All pass.

Throughput implication: the Lux network's aggregate throughput is bounded below by the slowest DAG chain and above by the sum. With GPU ecrecover + GPU keccak + GPU bitmap-intersect shared across all chains (single context per host), adding chains is strictly additive.

14 Conclusion

We have shown that the Lux multi-chain admits a uniform DAG consensus model in which the execution engine's conflict graph *is* the consensus DAG, yielding a single implementation pattern that covers EVM, UTXO, ZK-UTXO, DEX, Oracle, Relay, AI, and Service-Node workloads. Safety, serialisability, and double-spend prevention are proved in Lean 4 and model-checked in TLA$^+$.

The GPU EVM stack delivered through tags `v0.22.0` and `v0.23.0` of the `luxcpp/evm` repository extends the critical-path argument: ecrecover and keccak retain their original 32.3× and 7.1× speedups, the Metal kernel is opcode-complete on Cancun, the CUDA port reproduces the Metal kernel under a four-way parity test (133 hand-crafted bytecode vectors, four independent backends, bit-exact agreement on gas/status/output), and 100% function coverage on the kernel CPU spec witness ensures every implemented opcode is exercised on every CI run. Three consensus-relevant bugs (mulmod carry-loss, unrecognized-opcode gas accounting, MSTORE offset overflow) were surfaced by the differential-oracle property of cross-implementation parity and fixed in both kernels.

The formal development is open-source at `lux/formal`; the benchmarks are at `lux/evm-bench` and `lux/benchmarks`; the Go implementation is at `lux/evmgpu` and `lux/chains`; the GPU EVM kernel and parity test driver are at `luxcpp/evm` under tag `v0.23.0`.

References

[1] R. Gelashvili, A. Spiegelman, Z. Xiang, G. Danezis, Z. Li, D. Malkhi, Y. Xia, R. Zhou. *Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing.* PPoPP 2023.

[2] K. Babel, A. Chursin, G. Danezis, L. Kokoris-Kogias, A. Sonnino. *Mysticeti: Reaching the Limits of Latency with Uncertified DAGs*. arXiv:2310.14821.

[3] J. Tang et al. *Monad: Decoupling Execution and Consensus for Parallel EVM*. Monad Labs whitepaper, 2024.

[4] V. Yin, M. Ho. *pevm: Parallel EVM*. Risechain whitepaper, 2024.

[5] Paradigm. *Reth: A Rust Ethereum Execution Layer Client*. 2024.

[6] Ipsilon. *evmone: Fast Ethereum Virtual Machine Implementation*.

[7] G. Danezis, L. Kokoris-Kogias, A. Sonnino, A. Spiegelman. *Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus*. EuroSys 2022.